Lab-2 Report: CNN Training Dynamics on CIFAR-10

Name: *Siddhartha Singh*
Roll Number: *B22BB041*
Github Branch Link
Wandb Run Report Link

---

1 Experimental Setup

A custom Convolutional Neural Network (CNN) was trained on the **CIFAR-10** dataset using a custom PyTorch DataLoader. CIFAR-10 consists of 60,000 RGB images of size $32 \times 32 \times 3$, divided into training and test sets.

The experimental configuration is summarized below:

- **Model:** Custom CNN with multiple convolutional blocks and fully connected layers
- **Dataset:** CIFAR-10
- **Optimizer:** Adam
- **Learning Rate:** Scheduled decay
- **Epochs:** 30
- **DataLoader:** Custom implementation
- **Logging:** Weights & Biases (W&B)

2 FLOPs Analysis

The FLOPs for the CNN model were computed directly from the implemented PyTorch model using the same forward-pass configuration as used during training. In the code, FLOPs were estimated by performing a single forward pass with a dummy input tensor matching the CIFAR-10 input shape ($1\times3\times32\times32$) and summing the floating-point operations across all convolutional and fully connected layers.

The FLOPs computation considers only the model architecture (kernel sizes, number of filters, feature map dimensions, and linear layers) and does not depend on the optimizer, batch size during training, or the compute device. As a result, the reported FLOPs remain identical for CPU and GPU executions.

It is important to note that the FLOPs reported correspond to the computational cost of a single forward pass of the network. Backward-pass operations, optimizer updates, and data loading overheads are not included in this calculation. Therefore, FLOPs represent a theoretical measure of model complexity rather than actual training time.

This implementation-specific approach explains why the FLOPs values remain constant across epochs and hardware settings, while training time varies significantly between CPU and GPU executions.

3 Gradient Flow Analysis

Gradient flow visualizations were generated at Epochs 1, 5, 10, 15, 20, and 25 to analyze how gradients propagate through the network during training. For each layer, both the maximum and mean gradient magnitudes were recorded.

Early Training (Epoch 1)

At Epoch 1, very large gradient magnitudes are observed in the final fully connected layer, while early convolution layers also receive strong gradients. Intermediate convolution layers exhibit smaller but non-zero gradients. A large gap between maximum and mean gradients indicates high gradient variance during early optimization.

Mid Training (Epochs 5–15)

During mid training, gradient magnitudes decrease steadily across all layers. The final fully connected layer consistently exhibits the highest gradients, while convolution layers maintain stable and non-vanishing gradients. No exploding or vanishing gradients are observed.

Late Training (Epochs 20–25)

In later epochs, gradients become uniformly small across all layers. Mean gradients are extremely small, indicating fine-grained parameter updates. No sudden spikes or inactive layers are observed, suggesting near-converged training.

4 Weight Update Flow Analysis

Weight update flow plots show the mean magnitude of parameter updates per layer at Epochs 1, 5, 10, 15, 20, 25, and 30.

Early Training (Epoch 1)

Large weight updates are observed across most convolutional and fully connected layers, indicating rapid learning from random initialization.

Mid Training (Epochs 5–15)

Weight update magnitudes decrease noticeably. Deeper convolution layers exhibit relatively higher updates compared to shallow layers, while fully connected layers begin stabilizing earlier.

Late Training (Epochs 20–30)

Weight updates drop to very small magnitudes (as low as 10^{-5}). Updates become uniform across layers, and fully connected layers show minimal change, confirming smooth convergence.

5 Combined Observations

Gradient flow and weight update flow analyses are consistent with each other. Large gradients early in training correspond to large weight updates, while reduced gradients in later epochs result in smaller updates. The training dynamics indicate stable optimization and effective convergence.

6 Logging and Reproducibility

All relevant metrics and visualizations were logged to **Weights & Biases (W&B)**, including:

- Gradient flow plots at multiple epochs
- Weight update flow plots at multiple epochs

- Training and validation loss curves
- Accuracy curves
- Learning rate schedule

7 Conclusion

The CNN trained on CIFAR-10 demonstrates stable gradient propagation, controlled weight updates, and smooth convergence over 30 epochs. No signs of vanishing or exploding gradients were observed. Both gradient flow and weight update flow analyses confirm that the training process is numerically stable and correctly implemented.